

POP — Desenvolvimento e Operação do Rastro Globus

Diagnóstico técnico do Rastro Globus

Auditoria baseada em leitura do repositório em 2026-10-05. Itens não encontrados no código estão marcados como **PENDENTE DE DEFINIÇÃO** ou **RECOMENDAÇÃO TÉCNICA**.

Stack do backend (CONFIRMADO)

Evidência	Conteúdo
backend/requirements.txt	Django>=5.0,<6; DRF; SimpleJWT; django-cors-headers; Pillow; oracledb; cryptography; mysqlclient; python-dotenv; openpyxl; openai; gunicorn
backend/Dockerfile	Imagem base python:3.12-slim-bookworm
backend/rastroglobus/settings.py	App apps.core; JWT; CORS; AUTH_USER_MODEL = core.Usuario
backend/apps/core/urls.py	Rotas /api/auth/*, garantias, globus, relatórios
Migrations	0001_initial ... 0004_parte_d_regras em backend/apps/core/migrations/
Commands	seed, migrate (Django), check_db, sync_globus, detectar_reincidencia, import_planilha, aceite_golive
Testes	backend/apps/core/tests/test_*.py (unittest/Django)

Stack do frontend (CONFIRMADO)

Evidência	Conteúdo
frontend/package.json	React 18, React Router 6, Vite 5, TypeScript 5, Tailwind 3, Recharts
Scripts npm	dev, build (tsc -b && vite build), preview
frontend/vite.config.ts	Dev server porta 5173 ; proxy /api e /media → http://localhost:8000
Páginas	Login, Dashboard, GarantiasList, GarantiaNova, GarantiaDanfe, GarantiaFicha, RelatorioRankings

Banco e persistência (CONFIRMADO)

Modo	Como
SQLite (local sem DB_ENGINE)	backend/db.sqlite3 — settings.py
MariaDB/MySQL	Via DB_ENGINE=django.db.backends.mysql + host/user/senha
Docker Compose	Serviço db = imagem mariadb:11.4; backend força DB_HOST=db
Schema app	Django migrations (não os arquivos em SQL/)
Media/static	Volumes Docker media_data, static_data; MEDIA_ROOT / STATIC_ROOT no Django

Pasta [SQL/](#) : scripts **SELECT/DDI Oracle Globus** (ERP externo). **Não** são migrations do RastroGlobus.

Serviços Docker Compose (CONFIRMADO)

Arquivo: [docker-compose.yml](#)

```
[Navegador :8888]
|
[web = nginx:1.27 + frontend build]
| /api → backend:8000
| /media,/static → volumes
|
[backend = gunicorn Django :8000]
|
[db = MariaDB 11.4 :3306 interno]
```

Serviço	Imagem/build	Porta publicada	Depende de	Healthcheck
db	mariadb:11.4	nenhuma (só rede interna 3306)	—	healthcheck.sh --connect --innodb_initialized (5s/5s/20)
backend	build backend/Dockerfile	nenhuma (8000 interno)	db healthy	—
web	build deploy/Dockerfile.web	8888→80	backend	—

Volumes nomeados: `db_data` , `media_data` , `static_data` . Rede: default do Compose (implícita).

Entrypoint backend ([backend/docker-entrypoint.sh](#)): aguarda MySQL → `migrate --noinput` → `collectstatic --noinput` → `gunicorn` (3 workers, timeout 120).

Dependências entre componentes (CONFIRMADO)

De → Para	Tipo
Frontend (dev) → Backend	Proxy Vite <code>/api</code> , <code>/media</code> → <code>:8000</code>
Frontend (Docker) → Backend	Nginx <code>proxy_pass /api/</code> → <code>http://backend:8000</code>
Backend → MariaDB	TCP <code>db:3306</code> no Compose; SQLite ou MariaDB host no local
Backend → Oracle Globus	Somente leitura (<code>oracledb</code> + pasta <code>conf/</code>) — opcional
Backend → OpenAI	OCR DANFE opcional (<code>OPENAI_API_KEY</code>)
SQL/ → Globus Oracle	Consultas manuais no ERP; fora do Compose

Não encontrado no repositório: fila (Redis/Rabbit), cache dedicado, service mesh, Kubernetes, CI/CD (GitHub Actions).

Variáveis de ambiente (CONFIRMADO — sem valores secretos)

Raiz `.env` (Compose) — [.env.example](#) :

Variável	Serviço / uso
SECRET_KEY	Django
DEBUG	Django
ALLOWED_HOSTS	Django
CSRF_TRUSTED_ORIGINS	Django
DB_NAME, DB_USER, DB_PASSWORD	MariaDB + Django
DB_ROOT_PASSWORD	MariaDB root (Compose)
OPENAI_API_KEY, OPENAI_VISION_MODEL	OCR DANFE
RASTROGLOBUS_CONF_DIR, ORACLE_CLIENT_LIB_DIR	Globus/Oracle
GLOBUS_CALL_TIMEOUT_MS, GLOBUS_GRUPOS_PECAS	Globus

Compose injeta no backend: `DB_ENGINE` , `DB_HOST=db` , `DB_PORT=3306` .

Backend local — [backend/.env.example](#) : mesmas chaves + `DB_ALLOW_LEGACY_MARIADB` , `DB_ENGINE/HOST/PORT` vazios (= SQLite).

Scripts disponíveis (CONFIRMADO)

Script / comando	Finalidade
<code>docker compose up -d --build</code>	Subir stack (README)
<code>docker compose exec backend python manage.py seed</code>	Seed usuários/dados
<code>python manage.py migrate / seed / runserver / check_db</code>	Local backend
<code>python manage.py sync_globus (+ flags)</code>	Sync espelho Globus
<code>python manage.py detectar_reincidencia</code>	Alertas
<code>python manage.py import_planilha ...</code>	Import BASE GERAL
<code>python manage.py aceite_golive</code>	Checklist go-live
<code>npm run dev / build / preview</code>	Frontend
scripts/sync_globus_job.ps1	Job Windows sync
scripts/register_sync_globus_task.ps1	Agenda Windows
scripts/import_planilha_real.ps1	Import planilha
scripts/aceite_golive_check.ps1	Wrapper go-live

Testes, lint, qualidade (CONFIRMADO / LACUNAS)

Item	Status
------	--------

Testes Django (apps/core/tests/)	Confirmado (3 módulos)
Comando de teste documentado no README	Não documentado
ESLint / Prettier / Ruff / Black	Não encontrados configs no repo
CI/CD (.github/workflows)	Ausente
Tipagem frontend	TypeScript no build (tsc -b)

Deploy (CONFIRMADO)

Evidência	Conteúdo
README	Host srv-af-des01:8888; código /var/www/rastroglobus; docker compose up -d --build
deploy/Dockerfile.web	Build Node 20 + nginx
deploy/nginx/default.conf	Proxy /api, static/media, SPA
Scripts *.py em deploy/	Ignorados pelo git (.gitignore: deploy/*.py) — não fazem parte do processo versionado

Homologação formal, pipeline CI e ambiente prod separado: **PENDENTE DE DEFINIÇÃO**.

Riscos e lacunas (auditoria)

Risco	Evidência
Senhas seed documentadas no README (MVP)	README “Usuários seed”
CORS_ALLOW_ALL_ORIGINS = True	settings.py
SECRET_KEY default inseguro se env vazio	settings.py
Sem CI — regressão só local	ausência .github/
Docs ops desatualizados vs UI (ex.: “Buscar no Globus” em operacao-diaria-novos-rg.md)	drift documental
yota-chefe-arquitetura.md é persona de agente, não arquitetura do produto	arquivo na raiz
SQL/ é Oracle Globus — risco de execução acidental no MariaDB	pasta SQL
Compose não monta conf/ Oracle por padrão	docker-compose.yml
Alterar DB_PASSWORD no .env após volume criado não altera senha no MariaDB	comportamento MariaDB

Comandos CONFIRMADOS vs RECOMENDAÇÃO

Confirmados no repositório (README / compose / package.json / manage.py):

```
cp .env.example .env
docker compose up -d --build
```

```
docker compose exec backend python manage.py seed
docker compose exec backend python manage.py check_db
docker compose ps
docker compose logs
docker compose down
cd backend && python manage.py migrate && python manage.py seed && python manage.py
runserver
cd frontend && npm install && npm run dev && npm run build
python manage.py sync_globus
python manage.py import_planilha ...
.\scripts\aceite_golive_check.ps1
```

RECOMENDAÇÃO TÉCNICA (não documentada como padrão oficial no repo):

```
# Executar testes Django
cd backend && python manage.py test apps.core

# Logs Compose
docker compose logs -f backend web db

# Recriar banco Docker (APAGA DADOS)
docker compose down -v
```

Sumário

- [1. Controle do documento](#)
- [2. Objetivo](#)
- [3. Escopo](#)
- [4. Arquitetura do sistema](#)
- [5. Papéis e responsabilidades](#)
- [6. Pré-requisitos](#)
- [7. Configuração local](#)
- [8. Execução local](#)
- [9. Banco de dados e SQL](#)
- [10. Fluxo de versionamento](#)
- [11. Pull Request e code review](#)
- [12. Testes e qualidade](#)
- [13. Segurança e gestão de segredos](#)
- [14. Build, release e versionamento](#)
- [15. Deploy](#)
- [16. Rollback e contingência](#)
- [17. Gestão de incidentes](#)
- [18. Logs, observabilidade e auditoria](#)
- [19. Onboarding técnico](#)
- [20. Melhorias priorizadas](#)

1. Controle do documento

Campo	Conteúdo
-------	----------

Nome	POP de Desenvolvimento e Operação — Rastro Globus
Versão	1.0.0
Data	2026-10-05
Projeto	Rastro Globus
Responsável técnico	PENDENTE DE DEFINIÇÃO
Revisão periódica	Trimestral ou após incidentes relevantes
Local do documento	docs/POP_DESENVOLVIMENTO_E_OPERACAO.md

Histórico de alterações

Versão	Data	Autor	Descrição
1.0.0	2026-10-05	Auditoria repositório	Criação inicial a partir do código e docs existentes

2. Objetivo

Padronizar o ciclo de vida técnico do RastroGlobus:

- Desenvolvimento e execução local (com e sem Docker).
- Controle de versão e revisão de código.
- Banco de dados (migrations Django) e cuidado com scripts Oracle em `SQL/`.
- Testes e build.
- Deploy via Docker Compose no host documentado.
- Rollback, incidentes, segredos e documentação operacional.

3. Escopo

Cobre

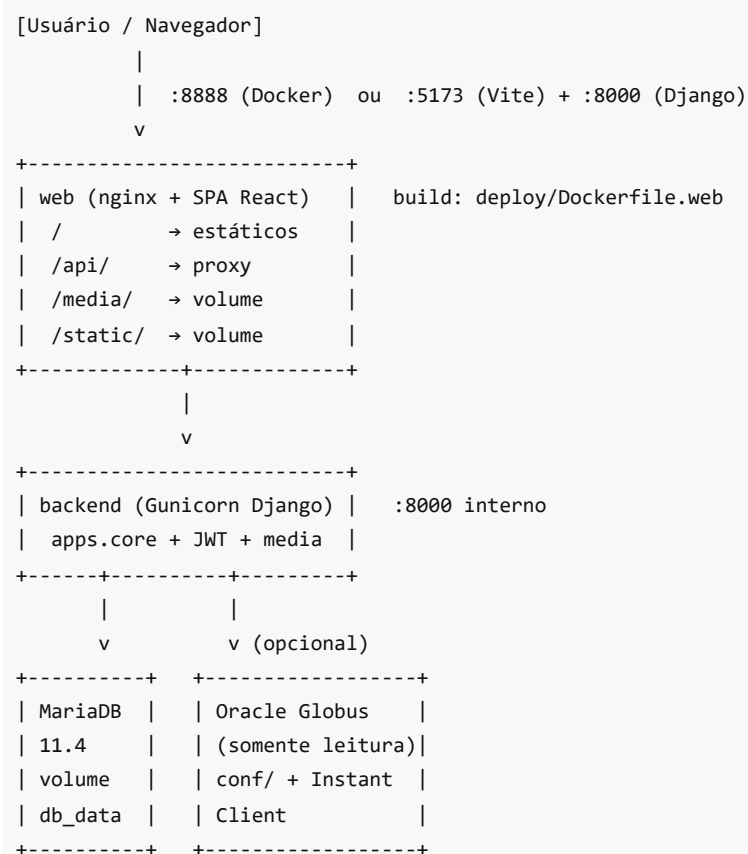
- Monorepo `backend/`, `frontend/`, `docker-compose.yml`, `deploy/`, `scripts/`, `docs/`, `SQL/`.
- Ambientes: desenvolvimento local; servidor de desenvolvimento documentado (`srv-af-des01:8888`).
- Operação Docker Compose (db + backend + web).
- Jobs Windows de sync/import/go-live descritos no README.

Não cobre

- Movimentação de estoque no ERP Globus (proibido pela regra de produto).
- Alteração de schema Oracle Globus via `SQL/` como parte do deploy do RG.
- Homologação/produção corporativa formal — **PENDENTE DE DEFINIÇÃO**.
- Pipeline CI/CD — **ausente no repositório**.
- Gestão de acessos AD/VPN — **PENDENTE DE DEFINIÇÃO**.

4. Arquitetura do sistema

Diagrama (CONFIRMADO)



Modo local sem Docker (CONFIRMADO no README):

```
[Vite :5173] --proxy--> [Django runserver :8000] --> [SQLite ou MariaDB host]
```

Responsabilidades por pasta

Componente	Responsabilidade
frontend/	UI React/Vite; autenticação JWT; telas de garantia/DANFE/relatórios
backend/	API Django/DRF; regras de garantia; sync Globus; OCR opcional
SQL/	Consultas Oracle Globus (suporte/ops ERP) — não schema do RG
scripts/	Automação Windows (sync, import, go-live)
deploy/	Dockerfile web + nginx; scripts <code>_*.py</code> não versionados
docs/	Ops cutover, operação diária, este POP
docs/ops/incoming/	Planilhas de import (xlsx ignorados pelo git)
conf/	Credenciais Oracle (não versionar — <code>.gitignore</code>)
docker-compose.yml	Orquestração db + backend + web
README.md	Entrada operacional principal

RASTROGLOBUS-CURSOR-SPEC.md	Spec de produto/MVP
yota-chefe-arquitetura.md	Persona de agente IA — não é arquitetura do sistema

5. Papéis e responsabilidades

Papel	Responsabilidades
Desenvolvedor	Implementação, testes locais, atualização de documentação afetada
Revisor	Code review, segurança, regressão, ausência de segredos no diff
Responsável por banco	Validar/migrations Django; NÃO executar <code>SQL</code> / Oracle no MariaDB do RG sem análise
Responsável por deploy	Aprovar e executar <code>docker compose</code> no host; validar pós-deploy
Responsável por incidente	Comunicação, mitigação, rollback, post-mortem SEV1/SEV2

Observação: se o projeto for operado por uma única pessoa, os papéis podem ser acumulados, mas os checklists deste POP e de [POP CHECKLIST OPERACIONAL.md](#) continuam obrigatórios.

6. Pré-requisitos

Ferramentas (CONFIRMADO / inferido de arquivos)

Ferramenta	Evidência	Versão confirmada
Git	fluxo de repositório	PENDENTE DE DEFINIÇÃO (versão mínima)
Docker + Compose	<code>docker-compose.yml</code> , README	Compose usado no README; versão mínima PENDENTE
Python	Dockerfile 3.12	3.12 no container; local PENDENTE alinhamento
Node.js	Dockerfile.web node:20-alpine	20 no build Docker; local PENDENTE
npm	<code>package.json</code> / <code>npm install</code>	—
Cliente MySQL/MariaDB	<code>check_db</code> , <code>mysqlclient</code>	opcional para ops
PowerShell	<code>scripts/*.ps1</code>	Windows para jobs Globus

Acessos

Acesso	Status
Clone do repositório GitHub	Confirmado remoto <code>origin</code> no histórico de uso
SSH ao <code>srv-af-des01</code> + Docker	Documentado operacionalmente no README

Pasta <code>conf/</code> Oracle	Necessária só se Globus ao vivo
OpenAI API	Opcional OCR
Credenciais seed	Documentadas no README (trocar em ambientes reais)

Checklist

- ☐ Git instalado
- ☐ Docker instalado
- ☐ Docker Compose disponível (`docker compose version`)
- ☐ Repositório clonado
- ☐ Arquivo `.env` criado a partir de `.env.example` (raiz para Docker **ou** `backend/.env` para local)
- ☐ Credenciais configuradas localmente **sem commit**
- ☐ Portas livres: **8888** (Compose), ou **5173 + 8000** (local)
- ☐ Serviços externos (Oracle/OpenAI) acessíveis, quando aplicável
- ☐ Instant Client + `conf/` se for usar Globus

7. Configuração local

Opção A — Docker Compose (caminho documentado para servidor/dev containerizado)

- Clonar o repositório.
 - Na raiz: `cp .env.example .env` (Linux/macOS) ou copiar manualmente no Windows.
 - Preencher variáveis (ver seção Diagnóstico) — **nunca** commitar `.env`.
 - Dependências: resolvidas **dentro** das imagens (pip/npm no build).
 - Banco: criado pelo serviço `db` na primeira subida (volume `db_data`).
 - Subir: `docker compose up -d --build`.
 - Seed (1ª vez): `docker compose exec backend python manage.py seed`.
 - Validar: `docker compose exec backend python manage.py check_db` e abrir `http://srv-af-des01:8888` (ou `http://localhost:8888` se mapeado localmente).
 - Desligar: `docker compose down`.
- ALERTA:** `docker compose down -v` apaga volumes (`db_data` , `media`, `static`) — perda de dados.

Opção B — Backend + Frontend sem Docker (CONFIRMADO no README)

- Clonar o repositório.
- `cd backend` → criar venv → `pip install -r requirements.txt`.
- Criar `backend/.env` a partir de `backend/.env.example`.
- Sem `DB_ENGINE` → SQLite; com MariaDB host → preencher `DB_*` e criar database utf8mb4.
- `python manage.py migrate` → `python manage.py seed` → `python manage.py runserver (:8000)`.
- `cd frontend` → `npm install` → `npm run dev (:5173)`.
- Validar login com usuário seed.
- Encerrar processos `runserver` / Vite; não há comando Compose nesta opção.

Categorias de variáveis (finalidade)

Categoria	Chaves	Finalidade
Django	SECRET_KEY, DEBUG, ALLOWED_HOSTS, CSRF_TRUSTED_ORIGINS	Segurança e hosts

Banco	DB_*, DB_ROOT_PASSWORD, DB_ALLOW_LEGACY_MARIADB	Persistência
Globus	RASTROGLOBUS_CONF_DIR, ORACLE_CLIENT_LIB_DIR, timeouts/grupos	Leitura Oracle
OCR	OPENAI_*	Extração DANFE por foto

8. Execução local

Fluxo Compose

1. Verificar: `docker compose ps` .
2. Subir: `docker compose up -d --build` (ordem: db healthy → backend → web).
3. Logs: `docker compose logs -f backend / web / db` (**RECOMENDAÇÃO TÉCNICA** de uso; comando Compose padrão).
4. Validar:
 - UI: HTTP na porta **8888**
 - API autenticada: `/api/` retorna 401 sem token (esperado)
 - Login: `POST /api/auth/login/`
 - `check_db` no backend
5. Parar: `docker compose down` .
6. Limpeza destrutiva: `docker compose down -v` — **somente com autorização**; apaga banco e media.

Tabela de serviços

Serviço	Responsabilidade	Comando para iniciar	Como validar	Diagnóstico
db	MariaDB 11.4	<code>docker compose up -d db</code>	healthcheck healthy; <code>check_db</code>	<code>docker compose logs db</code>
backend	API Gunicorn	sobe com up; endpoint <code>migrate+gunicorn</code>	<code>check_db</code> ; login API	<code>docker compose logs backend</code>
web	Nginx + SPA	sobe com up	HTTP 200 em <code>:8888/</code>	<code>docker compose logs web</code>
Django local	API dev	<code>python manage.py runserver</code>	<code>http://localhost:8000/api/</code>	stdout do processo
Vite local	UI dev	<code>npm run dev</code>	<code>http://localhost:5173</code>	terminal Vite

9. Banco de dados e SQL

Banco do RastroGlobus (CONFIRMADO)

- SQLite ou MariaDB/MySQL via Django.
- Schema: **somente** `python manage.py migrate` (migrations em `backend/apps/core/migrations/`).
- Seed: `python manage.py seed` (usuários e garantia exemplo).
- Validação: `python manage.py check_db` .

Pasta SQL/ (CONFIRMADO)

- Scripts voltados ao **Oracle Globus** (ex.: tipos de NF, acessos, relatórios de compras).

- **Não** executar esses arquivos no MariaDB do container `db` do RG.
- Uso: operação ERP / análise — **PENDENTE DE DEFINIÇÃO** o responsável e o ambiente Oracle alvo.

Alterações de schema no RG

1. Alterar models Django.
2. `makemigrations` / revisar migration gerada.
3. Aplicar com `migrate` em local/homologação antes de produção.
4. Commit da migration junto com o código.

Checklist obrigatório antes de alterar banco

- ☐ Script/migration revisado
- ☐ Backup confirmado (volume/ `db.sqlite3` /dump)
- ☐ Validado localmente ou em homologação
- ☐ Impacto analisado (downtime, locks)
- ☐ Plano de rollback documentado
- ☐ Janela de manutenção definida, se aplicável
- ☐ Resultado registrado após execução

Scripts destrutivos

- `docker compose down -v` , `DROP DATABASE` , truncate: exigem backup + dupla conferência + plano de reversão.
- Import planilha: preferir `--dry-run` antes do import real (README).

10. Fluxo de versionamento

CONFIRMADO: uso de Git + GitHub (`main`).

Branch `develOp` e tags: **PENDENTE DE DEFINIÇÃO** (não há política versionada no repo).

Abaixo: **RECOMENDAÇÃO TÉCNICA** compatível com equipe pequena.

Branches

Branch	Uso
<code>main</code>	Estável / deployável
<code>feature/nome</code>	Nova funcionalidade
<code>fix/nome</code>	Correção
<code>hotfix/nome</code>	Correção crítica em cima de <code>main</code>
<code>docs/nome</code>	Só documentação
<code>chore/nome</code>	Manutenção

`develop` : **PENDENTE DE DEFINIÇÃO** — só adotar se o time formalizar.

Conventional Commits (RECOMENDAÇÃO TÉCNICA)

```
feat: registrar foto DANFE obrigatória no cadastro
fix: corrigir tipagem labelVeiculo em GarantiaNova
docs: adicionar POP de desenvolvimento e operação
chore: ignorar deploy/*.py no gitignore
build: ajustar Dockerfile backend para LF no entrypoint
```

Exemplo de fluxo (não executar automaticamente)

```
git checkout main
git pull
git checkout -b feature/minha-mudanca
# ... alterações ...
git add <arquivos>
git commit -m "feat: descreve a mudança"
git push -u origin HEAD
# abrir PR no GitHub
```

11. Pull Request e code review

Template de PR (RECOMENDAÇÃO TÉCNICA)

```
## Objetivo
## Escopo técnico
## Componentes afetados
- [ ] frontend
- [ ] backend
- [ ] docker/deploy
- [ ] docs
## Alterações de banco
- [ ] Nenhuma
- [ ] Migration: (nome)
## Variáveis de ambiente
- [ ] Nenhuma
- [ ] Novas/alteradas: (listar chaves, sem valores)
## Evidências de testes
## Riscos conhecidos
## Plano de rollback
## Screenshots (se UI)
## Checklist do autor
- [ ] Sem segredos no diff
- [ ] `.env.example` atualizado se necessário
- [ ] Docs atualizados se necessário
```

Checklist do revisor

- ☐ Código compreensível e sem duplicação desnecessária
- ☐ Não há segredos, tokens ou senhas no diff
- ☐ Variáveis de ambiente foram documentadas

- ☐ Testes adequados foram executados
- ☐ Mudanças de banco foram revisadas
- ☐ API e frontend permanecem compatíveis
- ☐ Logs não expõem dados sensíveis
- ☐ Documentação foi atualizada quando necessário
- ☐ Impacto de performance foi considerado
- ☐ Plano de rollback existe para mudanças críticas
- ☐ Regra de negócio: RG **não** movimenta estoque Globus

12. Testes e qualidade

Tipo de validação	Comando confirmado	Quando executar	Critério de aprovação
Testes Django	RECOMENDAÇÃO: <code>python manage.py test apps.core</code> (arquivos em <code>apps/core/tests/</code> confirmados; comando não está no README)	Antes de PR / deploy	Todos passam
Tipagem + build frontend	<code>npm run build (tsc -b && vite build)</code>	Antes de PR / imagem web	Exit 0
Go-live técnico	<code>.\scripts\aceite_golive_check.ps1 / manage.py aceite_golive</code>	Antes de cutover	Conforme comando
Lint JS/Python	PENDENTE DE DEFINIÇÃO	—	—
CI automatizado	PENDENTE DE DEFINIÇÃO (ausente)	—	—
Teste manual UI	Login seed + fluxo DANFE/ficha	Antes de release	Fluxo crítico OK
check_db	<code>python manage.py check_db</code>	Após mudar DB	Imprime ok

13. Segurança e gestão de segredos

Regras obrigatórias

1. Nunca commitar `.env` real (já em `.gitignore`).
2. Nunca versionar `conf/chave.key`, `*.dat`, planilhas em `docs/ops/incoming/*.xlsx`.
3. Manter `.env.example` só com placeholders.
4. Não commitar `deploy/_*.py` com credenciais (já ignorados).
5. Rotacionar imediatamente qualquer segredo exposto.
6. Seed passwords do README são para **MVP/dev** — trocar em ambientes compartilhados (**RECOMENDAÇÃO**).
7. Revisar logs (Gunicorn/nginx) para não vazarem PII/credenciais.

Resposta a segredo vazado

1. Revogar/rotacionar o segredo (DB, OpenAI, Oracle, SSH).

- 2. Avaliar impacto (acesso indevido, dados).
- 3. Remover da aplicação/infra e de arquivos locais.
- 4. Remover do histórico Git se chegou a ser commitado.
- 5. Atualizar variáveis nos ambientes (`.env` servidor).
- 6. Registrar incidente (SEV1 se credencial de produção).
- 7. Ação preventiva (gitignore, review, secret scanning — **RECOMENDAÇÃO**).

14. Build, release e versionamento

Confirmado

Artefato	Como gerar
Frontend produção	<code>npm run build</code> → <code>frontend/dist</code> (também no estágio Docker <code>Dockerfile.web</code>)
Backend imagem	<code>docker compose build backend / build no up --build</code>
Web imagem	<code>build deploy/Dockerfile.web</code>
Static Django	<code>collectstatic</code> no entrypoint

PENDENTE DE DEFINIÇÃO

- Política de tags Git (`v1.2.3`).
- Changelog formal.
- Ambiente de homologação.
- Aprovação de release por papel.

Checklist pré-release (RECOMENDAÇÃO TÉCNICA)

- ☐ `main` atualizado
- ☐ Migrations incluídas
- ☐ `.env.example` alinhado
- ☐ `npm run build` OK
- ☐ Testes Django OK
- ☐ `check_db` OK no alvo
- ☐ Plano de rollback escrito
- ☐ Registro: versão, data, responsável, mudanças, riscos

15. Deploy

Ambiente documentado (CONFIRMADO)

Item	Valor
Host	<code>srv-af-des01</code>
URL	<code>http://srv-af-des01:8888</code>
Path código	<code>/var/www/rastroglobus</code>
Método	Docker Compose

Homologação / produção corporativa: **PENDENTE DE DEFINIÇÃO**.

Pré-requisitos

- Docker + permissão de socket para o usuário operador.
- `.env` na raiz do projeto no servidor (não versionado).
- Portas: **8888** publicada no host.

Sequência operacional (CONFIRMADO no README)

1. Atualizar código em `/var/www/rastroglobus` (git pull ou processo equivalente — **PENDENTE DE DEFINIÇÃO** o procedimento formal de sync de código).
2. Conferir `.env` (`SECRET_KEY` , `DB_*` , `ALLOWED_HOSTS` , `CSRF_TRUSTED_ORIGINS`).
3. `docker compose up -d --build` .
4. Se 1ª vez ou banco vazio: `docker compose exec backend python manage.py seed` .
5. Validar `check_db` , UI :8888 , login.

Validação pré-deploy

- ☐ Backup do volume/db se mudança destrutiva
- ☐ Migrations revisadas
- ☐ Sem segredos no pacote
- ☐ `.env` presente e correto

Validação pós-deploy

- ☐ `docker compose ps` — serviços Up/Healthy
- ☐ Home HTTP 200
- ☐ Login API OK
- ☐ `check_db` OK
- ☐ Smoke fluxo garantia/DANFE

Abortar deploy

- Build falha (ex.: `tsc`)
- Backend em restart loop
- `check_db` falha
- UI 502 persistente em `/api/`

Observabilidade

- `docker compose logs backend|web|db`
- Access/error do Gunicorn (stdout)

16. Rollback e contingência

Cenário	Sinal	Ação imediata	Rollback	Validação	Escalonamento
Falha de deploy (build)	<code>compose up --build exit ≠ 0</code>	Não promover; corrigir código	Manter containers anteriores se ainda Up	<code>compose ps</code>	Dev

API crítica	5xx / restart loop	docker compose logs backend	PENDENTE DE DEFINIÇÃO: tag/imagem anterior versionada; interim: restaurar código anterior + up --build	login + check_db	Resp. deploy
Frontend indisponível	:8888 falha	logs web	Rebuild web / restaurar imagem	HTTP 200 /	Dev
Banco inacessível	check_db falha; backend restart	Verificar db healthy e senhas .env vs MariaDB	Restaurar volume/backup — PENDENTE procedimento formal de backup	check_db	Resp. banco
Migration problemática	erro no entryptoint migrate	Parar backend; não forçar	Migration reverse se existir; senão restore backup	migrate + check_db	Resp. banco
Container em loop	STATUS Restarting	logs; corrigir entryptoint/CRLF/env	compose up com correção	ps Up	DevOps
Segredo vazado	commit/log/chat	Rotacionar já	Ver §13	acessos revogados	SEV1
Oracle Globus fora	badge/status sync falha	App continua com espelho local	N/A estoque; desligar sync	endpoints locais	Ops Globus
Script SQL Oracle errado	erro no cliente Oracle	Não aplicar no MariaDB RG	Rollback no Oracle conforme DBA	—	DBA/ERP

Lacuna: não há registry de imagens versionadas nem runbook de restore de `db_data` versionado no repositório.

17. Gestão de incidentes

Severidade	Descrição	Exemplo	Tempo de resposta
SEV1	Sistema indisponível ou risco crítico de segurança	App inoperante; credencial exposta	Imediato
SEV2	Função essencial gravemente afetada	Não cria garantia/DANFE; DB down	Prioridade alta

SEV3	Impacto moderado com alternativa	Relatório falha; Globus offline	Planejado
SEV4	Melhoria / impacto baixo	Ajuste visual	Backlog

Fluxo

1. Detectar e registrar.
2. Classificar severidade.
3. Mitigar impacto (rollback/feature flag/comunicar).
4. Comunicar responsáveis (lista: **PENDENTE DE DEFINIÇÃO**).
5. Corrigir ou aplicar rollback.
6. Validar recuperação (checklist pós-deploy).
7. Post-mortem para SEV1 e SEV2.
8. Atualizar este POP com aprendizados.

18. Logs, observabilidade e auditoria

Confirmado

Fonte	Como
Backend	stdout Gunicorn (docker compose logs backend)
Web/nginx	docker compose logs web
DB	docker compose logs db
Sync Globus (Windows)	logs em logs/ (gitignore) via script job
Domínio	EventoGarantia timeline imutável na aplicação

Cuidados

- Não logar senhas, tokens JWT, conteúdo completo de DANFE/PII desnecessário.
- `access_log off` em `/media` e `/static` no nginx (confirmado).

Métricas recomendadas (RECOMENDAÇÃO TÉCNICA)

- Disponibilidade `:8888` e taxa 5xx `/api/`
- Tempo de resposta Gunicorn
- Health MariaDB
- Último `SyncLog` Globus (`/api/globus/status/`)
- Espaço volumes Docker

Checklist mensal

- ☐ Revisar restarts de containers
- ☐ Backup/restore testado (quando processo existir)
- ☐ Rotação de senhas se política exigir
- ☐ Drift docs vs UI
- ☐ Dependências críticas (CVE) — processo PENDENTE

19. Onboarding técnico

Objetivo do primeiro dia:

1. Clonar o repositório.
2. Criar `.env` (raiz e/ou `backend/.env`).
3. Subir serviços (Compose **ou** runserver+Vite).
4. Acessar a aplicação (8888 ou 5173).
5. Rodar validações (`check_db` , preferencialmente testes).
6. Criar branch `feature/...`.
7. Fazer alteração simples (docs ou UI).
8. Abrir PR com template.

Checklist onboarding

- ☐ Leu README + regra “não movimenta estoque”
- ☐ Ambiente sobe sem erro
- ☐ Login seed funciona
- ☐ Entendeu diferença `SQL/` (Oracle) vs migrations Django
- ☐ Sabe onde está o POP (`docs/POP_*.md`)
- ☐ Não commitou `.env` / `conf/`

20. Melhorias priorizadas

Prioridade	Problema/Lacuna	Evidência	Risco/Impacto	Recomendação	Esforço
P0	Sem CI — quebras de <code>tsc</code> /testes só descobertas no build remoto	ausência <code>.github/workflows</code>	Deploy quebrado (já ocorreu com TS)	Pipeline: <code>npm run build + manage.py test</code>	Médio
P0	Backup/restore de <code>db_data</code> não documentado	volumes Compose sem runbook	Perda de dados em <code>down -v</code>	Script <code>dump/restore</code> MariaDB + teste periódico	Médio
P0	Documentação ops desatualizada (Buscar Globus etc.)	<code>docs/ops/operacao-diaria-novos-rg.md</code> vs UI atual	Erro operacional	Atualizar ops diária alinhada à tela DANFE	Baixo
P1	CORS aberto e <code>SECRET_KEY</code> default	<code>settings.py</code>	Superfície de ataque	Restringir CORS; exigir <code>SECRET_KEY</code> em <code>DEBUG=0</code>	Baixo
P1	Senhas seed no README em ambientes compartilhados	README	Acesso indevido	Forçar troca no 1º login / seed só em <code>DEBUG</code>	Baixo

P1	Compose sem montagem conf/ Oracle	docker-compose.yml	Globus indisponível no container	Volume opcional conf + Instant Client	Médio
P1	Política de branches/tags não formalizada	só main observada	Releases opacos	Adotar tags semver + este POP §10/14	Baixo
P2	Sem ESLint/Prettier/Ruff	configs ausentes	Drift de estilo	Adicionar linters mínimos	Médio
P2	SQL/ sem README de aviso	pasta SQL	Execução no DB errado	README: "somente Oracle Globus"	Baixo
P2	Homologação formal	PENDENTE	Risco direto em des→prod	Ambiente staging + checklist	Alto

Fim do POP v1.0.0 — Checklist operacional resumido: [POP_CHECKLIST_OPERACIONAL.md](#) .